Computación Concurrente, Paralela y Distribuida, Práctica 2

Concurrencia y compartición de datos con hilos

Versión: 3 de febrero de 2026

Contexto

Antes de empezar a manejar conceptos de paralelismo, o de trabajar con programas auténticamente paralelos, es interesante tener unas nociones de *concurrencia*. En nuestro ámbito, la concurrencia consiste en la gestión de **tareas que dependen entre ellas**, pero que no tienen una necesidad real de usar paralelismo, o que **no** están siquiera **pensadas para ejecutarse en paralelo**. Para calificar a un programa como concurrente, típicamente se ha considerado que tiene que estar compuesto de diversas unidades de ejecución (i.e., tareas), habitualmente repartidas en diversos *hilos de ejecución* (i.e., threads), pero que permite su ejecución de forma secuencial, o lo que es lo mismo, en 1 sola unidad de procesamiento (1 núcleo o 1 procesador mono-núcleo).

Durante la ejecución de este programa concurrente será el **Sistema Operativo** el que **alternará** entre unos **hilos** y otros cuando se produce un **evento bloqueante** durante la ejecución de una tarea, y que por ello provoca que el hilo tenga que abandonar el procesador. Es importante destacar que esto a su vez implica que la implementación del programa no puede presuponer ningún orden de ejecución exacto, ya que no solo hay una cierta aleatoriedad a la hora de escoger entre hilos del mismo programa a la hora de ser ejecutados, sino que el SO tiene que escoger también entre los hilos o procesos del programa concurrente y otros procesos o hilos de otros programas del sistema.

La **concurrencia** ha sido **históricamente muy usada** al no requerir hardware con capacidades de paralelismo (i.e., múltiples cores), algo que era esperable en la mayoría de computadores hasta hace relativamente poco tiempo en los **procesadores monocore**. Esto es debido a que en su esencia, la concurrencia busca aumentar el *aprovechamiento del tiempo de CPU* usando la ventaja de que el problema a resolver está dividido en diversas tareas, algunas de las cuales además van a requerir pasar tiempo fuera de la CPU en algún momento (esperas y eventos bloqueantes). Durante estos tiempos que serían de otra manera 'perdidos', se aprovechará para ejecutar otros hilos del mismo programa, si es posible.

De todas formas, con los equipos actuales, que disponen de múltiples unidades de procesamiento, en ocasiones será posible ejecutar estos programas de forma paralela sin que cambie idealmente el resultado final, algo que no es raro que el SO haga de forma natural al ver que el programa concurrente se compone de varios hilos.

Finalmente, es importante destacar que la gestión de tareas dependientes propia de la **concurrencia** es no obstante una potencial **fuente de problemas**, requiriendo así a menudo llevar a cabo un cuidadoso análisis y diseño previo para estos programas, algo que tiende a limitar su complejidad y capacidades.

Objetivos

En esta práctica usaremos una serie de códigos que implementan programas concurrentes con varios objetivos:

- Empezaremos entendiendo las **diferencias** entre el manejo de **variables compartidas** entre **procesos e hilos**, así como qué son los **locks**, la tecnología base para implementar programas concurrentes haciendo posible la coordinación y coherencia. De forma práctica veremos qué pasa cuando no se hace uso de dichos locks y hay una falta de coordinación.
- Ampliaremos el estudio con un **problema clásico** y aplicable a muchos escenarios reales, el de los Productores y Consumidores. En este escenario habrá una serie de entidades que serán ***productoras***, es decir que **generarán datos** que tendrán que ser procesados o 'consumidos' de alguna manera, y habrá entidades que serán ***consumidoras***, que realizarán el **procesamiento** de dichos **datos**. Con este escenario veremos además numerosos **problemas y limitaciones** que afloran cuando se hace uso de la **concurrencia**.
- Veremos como es posible ampliar el uso de hilos pasando de dos a muchos mediante la **compartición de datos** y su **sincronización** usando una **estructura** en forma de **cola**, algo que aunque resolverá algunos problemas expuestos, también tendrá sus propias limitaciones.

Índice

1 Diferencia entre Hilos y Procesos **3**

2 Locks y operaciones atómicas **6**
2.1 Conteo básico con variable compartida e hilos 8

3 Productores y Consumidores **10**
3.1 Escenario base y locks 10
3.2 Deadlock software 12
3.3 Condición de carrera 13
3.4 Inanición software 14

4 Múltiples productores y consumidores **17**
4.1 Programación de los códigos 17
4.2 Experimentación 20

Índice de Tareas

Tarea 1: Compartición de variables en procesos e hilo 3
Tarea 2: Conteo básico, tiempos y validación 8
Tarea 3: Conteo básico, efecto de los tiempos de espera 8
Tarea 4: Conteo básico, repartición del trabajo 9
Tarea 5: Productores y Consumidores, definiendo locks en el ódigo 10
Tarea 6: Productores y Consumidores, arrancando hilos en el código 10
Tarea 7: Productores y Consumidores, estudio de las funciones *producer* y *consumer* 10
Tarea 8: Productores y Consumidores, ejecutando el código 11
Tarea 9: Productores y Consumidores, problema del deadlock 12
Tarea 10: Productores y Consumidores, problema de la condición de carrera 13
Tarea 11: Productores y Consumidores, problema de la inanición 15
Tarea 12: Múltiples Productores y Consumidores, número de productores/consumidores 20
Tarea 13: Múltiples Productores y Consumidores, tiempos mínimos y máximos 20
Tarea 14: Múltiples Productores y Consumidores, tamaño de la cola 20
Tarea 15: Múltiples Productores y Consumidores, tamaño de la carga 20

Atención: Todas las imágenes en las que se muestra una ejecución son orientativas y no reemplazan una ejecución propia, algo que se aconseja de cara a la preparación de la evaluación.

1. Diferencia entre Hilos y Procesos

Considerando que un problema al que queremos aplicar paralelismo se puede resolver típicamente usando hilos o procesos, es crucial entender la diferencia entre ambas tecnologías. Por un lado, los hilos son entidades que viven dentro de un proceso, pero que tienen sus recursos y flujo de ejecución propios. Por otro lado, en el caso de paralelismo con procesos, cada proceso es un proceso independiente de cara al SO. No obstante, es de interés destacar que una solución paralela puede incluso combinar procesos e hilos.

En todo caso hay una diferencia fundamental entre procesos e hilos en lo que respecta al manejo de la memoria y al entorno de ejecución, y más específicamente en lo que nos interesa a nosotros, en las variables del programa. Esta diferencia se deberá a la diferente manera en la que se gestionan y crean procesos e hilos. Mientras que los procesos se clonan y cargan su espacio de memoria y variables propias (*operaciones fork + exec [opcional]*), momento a partir del cual siguen su ejecución de forma independiente, los hilos aún cuando son creados, pueden seguir compartiendo información y datos a través de la memoria global del proceso original que los aloja.

Compartición de datos entre procesos: A pesar de que los procesos sean entidades aisladas entre ellas de forma natural, también es posible que compartan datos entre ellos mediante diversos mecanismos. Esta compartición de datos entre procesos la veremos a continuación de forma puntual, pero será la usada en la siguientes prácticas cuando trabajemos con el proceso como entidad para la paralelización.

Tarea 1 – Compartición de variables en procesos e hilo

En el *'codigo0-ProcesosVsHilos.py'* podrás ver empíricamente la diferencia entre procesos e hilos en lo que respecta al valor de las variables. En este código se inicializa una variable inicial y a continuación se lanzarán hilos y procesos en 3 escenarios distintos. Tanto hilos como procesos reportarán el valor de la variable que ven, y lo cambiarán, pero lo harán con un desfase de 1 segundo entre ellos. Los distintos escenarios son:

- ***Escenario 1***: Se lanzan procesos, cada uno ha heredado una copia de la variable original, de ahí que todos reporten su mismo valor inicial, pero luego cada cambio que le hagan solo lo verán cada proceso de forma aislada. → **Los procesos NO comparten variable**
- ***Escenario 2***: Se lanzan hilos, cada uno tiene la misma referencia a la variable global, pero debido al desfase, solo el 1º reportará el valor original, el resto verán el valor del hilo anterior y lo cambiarán para que el nuevo valor sea visto por el hilo siguiente. → **Los hilos SI comparten variable**
- ***Escenario 3***: Se lanzan procesos, pero esta vez la variable ha sido configurada como variable global en una zona de memoria compartida entre procesos. Al igual que en con el caso de los hilos, el 1º proceso verá el valor original y los cambios serán propagados a los procesos siguientes. → **Los procesos SI comparten variable**

Ejecuta el código y observa los diferentes escenarios.

Programación funcional: Puedes aprovechar este primer código para observar el uso de la programación funcional en un ámbito de paralelización. Estudia las siguientes líneas del código0, en este orden:

- 65 → En esta línea se define la firma de una función ('spawn'), siendo el primer y único parámetro una función, no una variable. Esta función corresponderá a una función Python de paralelización, en este caso para lanzar hilos (*threading.Thread*) o procesos (*multiprocessing.Process*).
- 71 → En esta línea se usará la función pasada como argumento para llamarla de forma explícita. De igual forma, para llamar a esta función se le pasará como primer argumento (*target=*) otra función (en este caso la '*poll*'), que será la ejecutada por el hilo o proceso. Como segundo argumento se le pasará los argumentos que recibirá dicha función '*poll*'.
- 91, 95 → En estas líneas se llama a la función '*spawn*' definiendo, ahora sí, si queremos ejecutarla con hilos (función '*Thread*') o con procesos (función '*Process*').

Diferencias entre Hilos y Procesos (codigo0-ProcesosVsHilos.py)

```
49 def poll(delay_sleep, am_i_last, repeat_n_times):
50     i = 1
51     time.sleep(delay_sleep)
52     while i <= repeat_n_times:
53         prev_value = get_VARIABLE_value()
54         set_VARIABLE_value(random.randint(1, 100))
55         print("ENTITY {0}, number was {1}, "
56               "now it is {2}".format(os.getpid(), prev_value, get_VARIABLE_value()))
57         if am_i_last:
58             print("------------")
59         time.sleep(NUM_WORKERS)
60         i += 1
61
62 # Lanza los workers, que pueden ser procesos o hilos
63 # El argumento "parallelism_option" es una librería, puede ser la que arranca hilos o procesos,
64 # ambas tienen la misma firma
65 def spawn(parallelism_option):
66     worker_list = list()
67     am_i_last = False
68     for t in range(0, NUM_WORKERS):
69         if t == NUM_WORKERS - 1:
70             am_i_last = True
71         c = parallelism_option(target=poll, args=(t, am_i_last, CYCLE_N_TIMES,))
72         worker_list.append(c)
73
74     for w in worker_list:
75         w.start() # Arranca el hilo o proceso
76
77     for w in worker_list:
78         w.join() # Espera a que el hilo o proceso termine
```

```
Started main process
Going to use parallelism with 2 entities
Setting variable value with random value = 24

Spawning processes (Escenario 1)
ENTITY 22868, number was 24, now it is 85
ENTITY 22869, number was 24, now it is 50
------------
ENTITY 22868, number was 85, now it is 72
ENTITY 22869, number was 50, now it is 91
------------
Spawning threads (Escenario 2)
ENTITY 22867, number was 24, now it is 84
ENTITY 22867, number was 84, now it is 96
------------
ENTITY 22867, number was 96, now it is 7
ENTITY 22867, number was 7, now it is 31
------------
Spawning processes with shared variable (Escenario 3)
ENTITY 22891, number was 24, now it is 79
ENTITY 22892, number was 79, now it is 20
------------
ENTITY 22891, number was 20, now it is 62
ENTITY 22892, number was 62, now it is 56
------------
```

Figura 1: Ejecución del codigo0 con secuencia de modificación de variable en hilos y procesos

2. Locks y operaciones atómicas

Considerando el último escenario de la tarea anterior, hay que destacar que el uso de una variable global compartida puede no ser una buena idea, especialmente si está pensada para ser leída y escrita por varios procesos sin un orden específico.

En el caso anterior estamos en un escenario muy simple con pocos procesos y con operaciones de lectura y escritura realizadas cada pocos segundos, por lo que no debería haber problemas. No obstante, en otro escenario en el que hubiera cientos de lectores y escritores trabajando contra una misma variable en espacios cortos de tiempo (ms), podrían surgir problemas de *coherencia*. Esto se debe a que el valor de la variable está en memoria, pero si varios procesos o hilos trabajan con ella simultáneamente, a la hora de actualizar su valor, solamente el último cambio se tendrá en cuenta (el resto de entidades han usado un valor 'viejo' y sus cambios se pierden al ser 'pisados' por el último). Puedes ver una representación gráfica de este escenario en las Figuras 2 y 3.

Para mitigar este problema, en los diseños de programas concurrentes **que usan hilos** se añaden lo que se llaman *'semáforos'*, unas variables 'especiales', normalmente numéricas, que permiten ser cambiadas *de forma atómica*. Esto quiere decir que cuando un proceso cambia esta variable, el resto de procesos lo puede ver de forma instantánea, **las variables semáforo o lock NO sufren problemas de coherencia**. Con el uso de estas variables, podemos proteger un segmento de código de tal forma que solo pueda ser ejecutado simultáneamente por un número predeterminado de hilos. Este segmento de código puede a su vez encargarse de la gestión de una variable compartida, estando así por ello protegida también de ser alterada por varios hilos simultáneamente.

Finalmente, una variable 'semáforo' puede estar configurada para permitir el paso de '$n > 1$' hilos, mientras que un *'lock'* es un 'semáforo' de $n = 1$, es decir solo deja pasar a 1 hilo de cada vez (sección abierta o cerrada, variable accesible o no accesible). En esta práctica nos centraremos en el uso de locks.

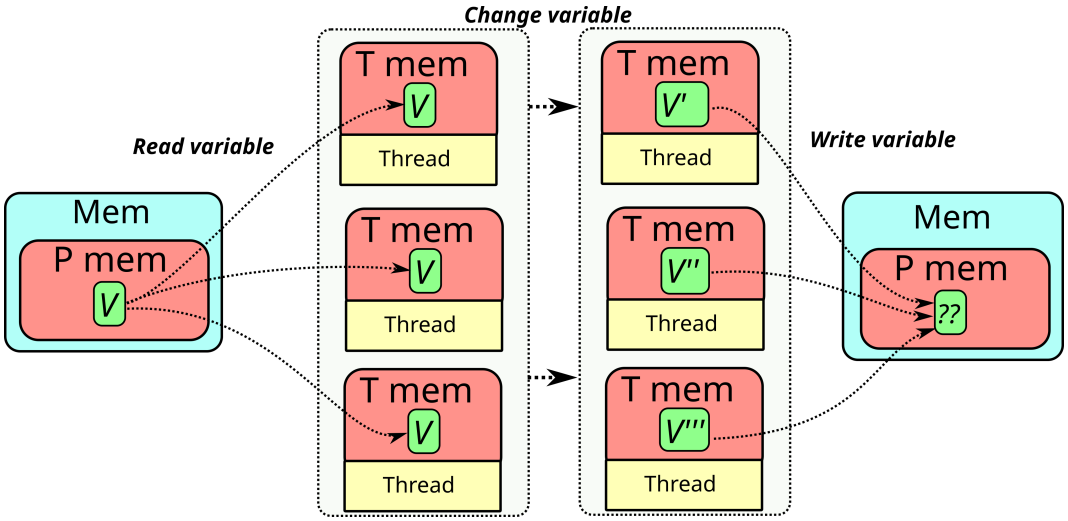

Figura 2: Secuencia de lectura y escritura de una variable por parte de diversos hilos

Operaciones atómicas: Una *operación atómica* (e.g., *acquire, release en Python*) es aquella que un programa realiza sin poder ser interrumpido, es decir, cuando la inicia, no puede ser desalojado del procesador de ninguna manera y la tiene que completar. Habitualmente estas operaciones son extremadamente cortas y simples, como incrementar o decrementar un contador. Su importancia radica en la atomicidad, de otro modo si dos procesos intentaran cambiar la variable semáforo, podría haber un problema de coherencia y por ello, perder su sentido de ser. Estas operaciones tienen por todo esto una implementación hardware de muy bajo nivel. Puedes ver un diagrama de su funcionamiento en la Figura 4.

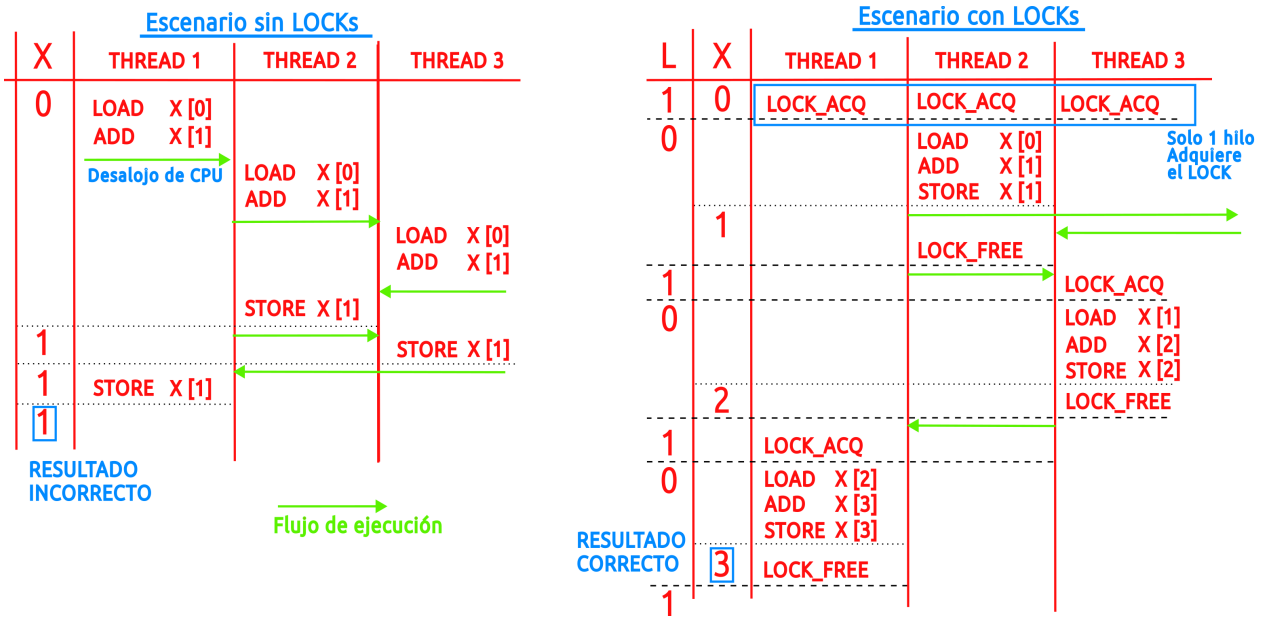

Figura 3: Explicación gráfica comparando el efecto de usar o no una variable de tipo lock

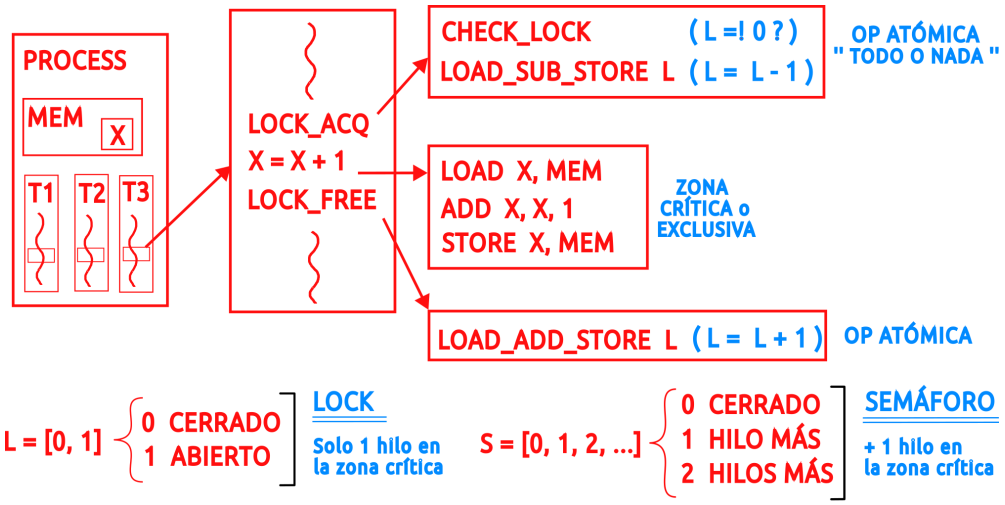

Figura 4: Explicación gráfica del funcionamiento de una instrucción atómica usada para implementar un lock

2.1. Conteo básico con variable compartida e hilos

Para poder ver el efecto del uso de estos locks para mantener la coherencia, vamos a ejecutar el *'codigo1-ThreadCount.py'*, en el que varios hilos trabajan de forma conjunta para llevar a cabo un conteo. En el programa hay un objeto, llamado 'CountContainer', que contendrá la cuenta. Dicha cuenta se inicializa en 0 y al finalizar el programa, debería ser igual a la variable **TO_BE_ADDED**. Este objeto tiene varios métodos que permiten leer la cuenta y fijar un nuevo valor, así como métodos para proteger el acceso a dicha cuenta si se desea, y un método para incrementar el valor de un número, el cual además simula un coste temporal variable (variable **MAX_WAIT_TIME_MS**).

Será el trabajo de todos los hilos el incrementar dicha cuenta, algo que tendrán que hacer de forma concurrente. Para ello hay varios métodos implementados, más específicamente dos denominados 'safe' y 'unsafe', que respectivamente hacen referencia a si los métodos se ejecutarán teniendo en cuenta que la variable es compartida y debe ser protegida con un acceso ordenado, o sin tener en cuenta esto. Puedes ver una representación gráfica en la Figura 5.

La salida de este código son dos tablas que reportan uno o más experimentos. La primera tabla consta de la configuración de las variables usadas y del tiempo en milisegundos para cada método, mientras que la segunda tabla refleja el valor final de la variable de conteo.

Conteo con múltiples hilos (codigo1-ThreadCount.py)

```
def unsafe(count, n):
    while n > 0:
        c = count.get_count()
        c = count.add_to_count(c, 1)
        count.set_count(c)
        n -= 1


def safe(count, n):
    while n > 0:
        count.lock_count()
        c = count.get_count()
        c = count.add_to_count(c, 1)
        count.set_count(c)
        count.unlock_count()
        n -= 1
```

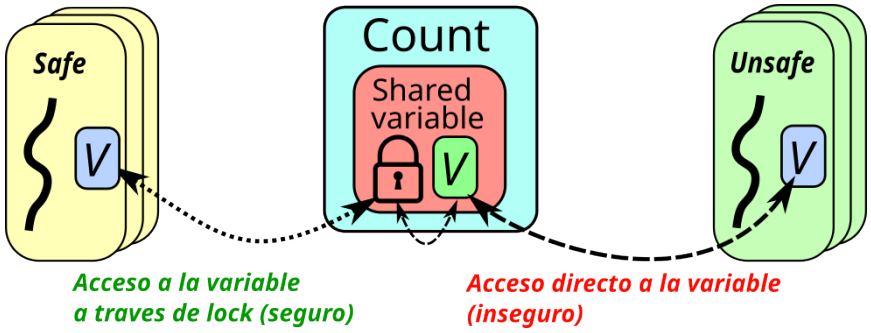

Figura 5: Diferencias entre conteo con hilos safe y unsafe

Tarea 2 – Conteo básico, tiempos y validación

Ejecuta el código con el escenario simple que encontrarás descomentado, y observa la salida.

- **¿Consiguieron llegar al conteo esperado todos los métodos y versiones?**
- **¿Qué método consiguió los mejores tiempos?**

```
alumno@ParalelismoMV:~/Compartida/P2-ConcurrenciaBasica$ python3 codigo1-ThreadCount.py
Running experiment with config: {'MAX_WAIT_TIME_MS': 200, 'TO_BE_ADDED': 200, 'N_THREADS': 20}
-----------------
TIME MEASUREMENTS (in ms)
   MAX_WAIT_TIME_MS  TO_BE_ADDED  N_THREADS   UNS    SAF
0               200          200         20  1621  23795
-----------------
FINAL COUNT VALUE
   MAX_WAIT_TIME_MS  TO_BE_ADDED  N_THREADS  UNS  SAF
0               200          200         20   10  200
```

Figura 6: Ejecución del codigo1

Tarea 3 – Conteo básico, efecto de los tiempos de espera

Ejecuta el código con los escenarios de experimentación que se encuentran comentados, a fin de ver el efecto de las variables **MAX_WAIT_TIME_MS** y **N_THREADS** en los dos métodos.

- **¿Qué efecto tiene la variable N_THREADS sobre los tiempos y la cuenta final?**
- **¿Y la variable MAX_WAIT_TIME_MS?**

Finalmente, es de interés también destacar la repartición del trabajo en este código. Para hacer dicha repartición se ha procedido a dividir la cuenta final deseada entre los hilos para que todos tengan más o menos el mismo trabajo. Dado que la cuenta puede ser indivisible, las cuentas parciales pueden ser distintas para cada hilo en base a si la variable **TO_BE_ADDED** es múltiplo o no de **N_THREADS**. Debido a esto, a parte de la división ('/') será interesante tener en cuenta la operación *modulo*('%').

Tarea 4 – Conteo básico, repartición del trabajo

Analiza la repartición del trabajo realizada en las siguientes líneas:

Repartición del trabajo entre los hilos (codigo1-ThreadCount.py)

```
    n = int(TO_BE_ADDED / N_THREADS)
    n_mod = TO_BE_ADDED % N_THREADS
    for thread_num in [*range(0, N_THREADS, 1)]:
        thread_slice = n
        if thread_num < n_mod:
            thread_slice += 1
```

Si la variable **TO_BE_ADDED** valiera 105 y la variable **N_THREADS** fuera 10:

- **¿sabrías decir la cuenta parcial que deben realizar los hilos?**
- **¿Tendría la misma cuenta parcial el hilo #0 y el #9?**

3. Productores y Consumidores

3.1. Escenario base y locks

El problema de los productores y consumidores es muy popular por su sencillez pero también por su aplicabilidad a múltiples problemas reales (e.g., consumir datos de sensores en IoT). En el *'codigo2-ProdCons.py'*, podrás ver la implementación de productores y consumidores pero en un sencillo escenario de 1 hilo productor y 1 hilo consumidor.

Tarea 5 – Productores y Consumidores, definiendo locks en el ódigo

Observa en la cabecera del programa las líneas:

Productores y Consumidores (codigo2-ProdCons.py)

```
23 item_ready = threading.Semaphore(0) # Esto por defecto está en 0
24 item_consumed = threading.Semaphore(1) # Esto por defecto está en 1
```

Estas líneas están definiendo 2 semáforos pero que luego serán por diseño usados como variables 'lock', y más concretamente uno de ellos se inicializará a 0 (lock cerrado) y el otro a 1 (lock abierto), acorde al uso que se les dará una vez el programa se arranque.

Tarea 6 – Productores y Consumidores, arrancando hilos en el código

Observa al final del fichero de código, en el cuerpo del 'main', las líneas 73–78:

Productores y Consumidores (codigo2-ProdCons.py)

```
73     t1 = threading.Thread(target=producer)
74     t2 = threading.Thread(target=consumer)
75     t1.start()
76     t2.start()
77     t1.join()
78     t2.join()
```

Estas líneas están creando 2 threads (líneas 73–74), uno para un productor que al ser arrancado ejecutará la función '*producer*' (*target=producer*), y otro para un consumidor que tendrá asociado la función '*consumer*' (*target=consumer*). De igual forma, también se arrancan los threads (líneas 75–76, función *.start()*) y se espera a que terminen (líneas 77–78, función *.join()*), concretamente a que termine primero el *producer*, y luego el *consumer*.

Tarea 7 – Productores y Consumidores, estudio de las funciones *producer* y *consumer*

Observa las funciones que serán ejecutadas por los hilos (producer y consumer), intenta averiguar su lógica interna. Como aclaratorio, hay que decir que la operación *.release()* es una **operación atómica** que incrementa el valor de la variable con la que se llama, en 1, mientras que la operación *.acquire()*, realiza el proceso inverso, decrementa en 1.

Productores y Consumidores (codigo2-ProdCons.py)

```
def consumer():
    # Una variable declarada como global hace referencia a una variable compartida entre todos los hilos
    global item, ITEMS_CONSUMED
    items_consumed = 0
    while True:
        consumer_notify("Waiting for new items")
        item_ready.acquire()
        consumer_notify("consuming item")
        time.sleep(SLEEP_TIME)
        items_consumed += 1
        consumer_notify("Consumed item number {0}".format(item))
        item_consumed.release()

        if items_consumed >= ITEMS_TO_CONSUME:
            ITEMS_CONSUMED = True
            break
```

Productores y Consumidores (codigo2-ProdCons.py)

```
def producer():
    global item, ITEMS_CONSUMED
    while True:
        producer_notify("Waiting to produce a new item")
        item_consumed.acquire()
        producer_notify("Producing an item")
        item = random.randint(0, 1000)
        producer_notify("Produced item number {0}".format(item))
        item_ready.release()

        if ITEMS_CONSUMED:
            break
```

Tarea 8 – Productores y Consumidores, ejecutando el código

Ejecuta ahora el código y observa la secuencia de eventos. Para entenderla, puedes apoyarte en la Figure 7, así como en la salida de su ejecución en la Figura 8.

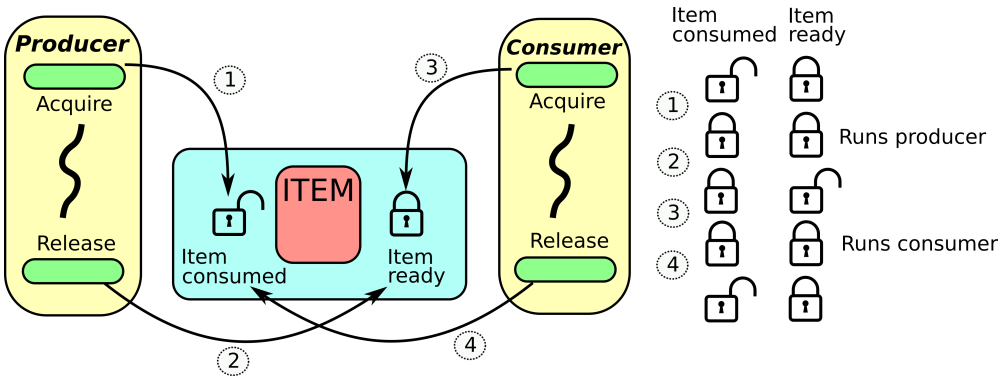

Figura 7: Representación gráfica de la secuencia de eventos en la ejecución del código1

Los locks como elementos bloqueantes: El quedarse esperando para acceder a una sección de código, es decir el quedarse bloqueado en una variable 'lock', es un evento bloqueante ya que habitualmente provoca que el thread se quede dormido. Esto es de interés de cara al aprovechamiento de la CPU por parte del SO.

```
Launching threads
Producer notify: [10:48:37] -> Waiting to produce a new item
Producer notify: [10:48:37] -> Producing an item
Producer notify: [10:48:37] -> Produced item number 946
Producer notify: [10:48:37] -> Waiting to produce a new item
                                                        Consumer notify: Waiting for new items
                                                        Consumer notify: consuming item
                                                        Consumer notify: Consumed item number 946
                                                        Consumer notify: Waiting for new items
Producer notify: [10:48:37] -> Producing an item
Producer notify: [10:48:37] -> Produced item number 610
Producer notify: [10:48:37] -> Waiting to produce a new item
                                                        Consumer notify: consuming item
                                                        Consumer notify: Consumed item number 610
                                                        Consumer notify: Waiting for new items
Producer notify: [10:48:37] -> Producing an item
Producer notify: [10:48:37] -> Produced item number 988
Producer notify: [10:48:37] -> Waiting to produce a new item
                                                        Consumer notify: consuming item
                                                        Consumer notify: Consumed item number 988
                                                        Consumer notify: Waiting for new items
Producer notify: [10:48:38] -> Producing an item
Producer notify: [10:48:38] -> Produced item number 726
Producer notify: [10:48:38] -> Waiting to produce a new item
                                                        Consumer notify: consuming item
                                                        Consumer notify: Consumed item number 726
                                                        Consumer notify: Waiting for new items
Producer notify: [10:48:38] -> Producing an item
Producer notify: [10:48:38] -> Produced item number 882
Producer notify: [10:48:38] -> Waiting to produce a new item
                                                        Consumer notify: consuming item
                                                        Consumer notify: Consumed item number 882
                                                        Consumer notify: Waiting for new items
```

Figura 8: Ejecución del código1 con secuencia de interacción con el lock

Los programas concurrentes a menudo hacen uso de múltiples hilos y recurren al uso de semáforos para la coordinación a la hora de ejecutar ciertas tareas, o a la hora de acceder y usar variables compartidas a fin de mantener su coherencia. No obstante, como previamente comentado, el uso de estas variables y en general el diseño de programas concurrentes con ellas requiere de un especial cuidado para evitar una serie de problemas, descritos a continuación.

3.2. Deadlock software

El *deadlock software*, también llamado *interbloqueo*, consiste en un abrazo mortal entre hilos en el cual ambos se quedan bloqueados mutuamente. Esta condición se puede dar fácilmente cuando hay una mala configuración de los locks que usan, pudiendo quedarse dependiendo el uno del otro. La gestión de interbloqueos es muy problemática y a menudo requiere intervención del exterior para resolverla.

Tarea 9 – Productores y Consumidores, problema del deadlock

Para generar una situación de deadlock software, cambia las variables semáforo del "codigo2-ProdCons.py", de tal manera que ambas tengan un valor inicial de 0. Ejecuta el programa y deberías ver como se queda 'congelado'.

- ¿Sabrías explicar el motivo?
- ¿Sabrías decir en qué líneas se han quedado parados los hilos?

En la Figura 9 puedes ver un diagrama de esta situación en comparación con el escenario anterior, así como la salida de una ejecución en la Figura 10.

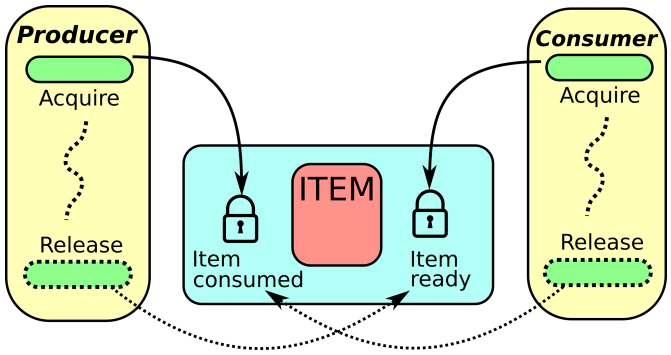

Figura 9: Representación gráfica de un deadlock

```
alumno@ParalelismoMV:~/Compartida/P2-ConcurrenciaBasica$ python3 codigo2-ProdCons.py
Launching threads
Producer notify: [10:43:30] -> Waiting to produce a new item
                                                          Consumer notify: Waiting for new items
```

Figura 10: Ejecución de un código con deadlock

Parada de los programas: Recuerda que puedes parar los programas mandando una señal de terminación al proceso maestro, que habitualmente es el que tiene su control asociado a la terminal que usas para lanzarlo. De igual forma, si presionas 'Control + C' en la terminal donde se está ejecutando el programa, estarás también enviando dicha señal. Al recibir la señal de terminación, el proceso maestro debería a su vez 'matar' a cualquier proceso o hilo hijos.

3.3. Condición de carrera

Otro problema habitual en los programas concurrentes, aunque también presente en los paralelos que veremos, son las *condiciones de carrera*. Este tipo de problemas surge cuando el orden de ejecución de unos procesos o hilos altera el resultado final, o dicho de otra forma, cuando tendremos un resultado final potencialmente distinto en cada ejecución porque el orden de ejecución no es determinista (no es conocido de antemano), y depende de factores externos como la gestión de procesos del SO.

Tarea 10 – Productores y Consumidores, problema de la condición de carrera

Observa el código *'codigo3-ProdConsRace.py'* y ejecútalo. Dado que no tiene 'final', puedes parar su ejecución con la combinación de 'Control + C'. Este código es similar al anterior, pero se ha simplificado para usar solamente 1 lock. En este código, el hilo productor no está coordinado con el consumidor e intentará producir aunque el resultado anterior no se haya consumido. De igual forma, el consumidor no está coordinado con el productor e intentará consumir siempre. Esta actitud se denomina en ocasiones 'voraces' ('*greedy*' en inglés). Puedes ver un diagrama de este escenario en la Figura 11, así como un ejemplo de su ejecución en la Figura 12.

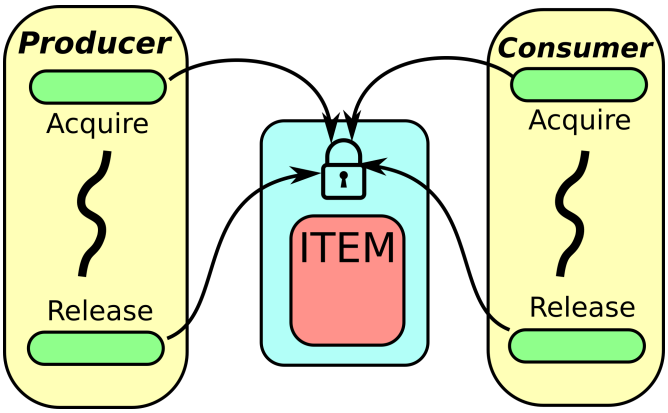

Figura 11: Representación gráfica de la configuración de lock que genera una condición de carrera

Carrera de hilos 'voraces' (codigo3-ProdConsRace.py)

```
def consumer():
    global item, won_lock_t2, FINISH
    while not FINISH:
        consumer_notify("Waiting for new items")
        item_ready.acquire()
        won_lock_t2 += 1
        consumer_notify("Entering production bay")
        consumer_notify("Consuming item")
        time.sleep(SLEEP_TIME)
        consumer_notify("Consumed item number {0}".format(item))
        item_ready.release()

def producer():
    global item, won_lock_t1, FINISH
    while not FINISH:
        producer_notify("Waiting to access production bay")
        item_ready.acquire()
        won_lock_t1 += 1
        producer_notify("Entering production bay")
        producer_notify("Producing an item")
        item = random.randint(0, 1000)
        time.sleep(SLEEP_TIME)
        producer_notify("Produced item number {0}".format(item))
        item_ready.release()
```

Si observas la salida del programa, podrás ver como la ejecución 'salta' de uno a otro hilo sin aparente orden, manteniéndose mientras en 1 solo hilo. Cuando un hilo consigue entrar en la zona de código protegida por el lock, puede mantenerse en ella y por ello en ejecución si logra soltar y adquirir de nuevo el lock, es decir si entre que lo suelta y lo adquiere el SO no decide poner en ejecución el otro hilo. Cuando esto ocurre y por ello pierde el lock, se produce ese 'salto' y será el otro hilo el que se encuentre en ejecución. La 'carrera' se genera por todo esto en la adquisición del lock. Al final, al estar casi en igualdad de condiciones, ambos threads deberían, por probabilidad, ganar el lock un número similar de veces.

3.4. Inanición software

Derivado del problema anterior puede surgir el problema de la *inanición*. En el caso anterior pudimos observar como ambos hilos 'mas o menos' podían ejecutarse, ya que en algún momento 'ganaban' la carrera y se hacían con el control de la zona de código protegida (tanto para consumir como para producir).

Sin embargo, si estamos en un escenario en el que los hilos tienen una lógica de ejecución diferente, y el tiempo que usan para adquirir el control es distinto, es decir si no están siempre atentos o 'corren la carrera más despacio',

```
[29: 8] -> Consumer notify: Entering production bay
[29: 8] -> Consumer notify: Consuming item
[29: 8] -> Consumer notify: Consumed item number 583
[29: 8] -> Consumer notify: Waiting for new items
[29: 8] -> Consumer notify: Entering production bay
[29: 8] -> Consumer notify: Consuming item
[29: 8] -> Consumer notify: Consumed item number 583
[29: 8] -> Consumer notify: Waiting for new items
[29: 8] ->                                          Producer notify: Entering production bay
[29: 8] ->                                          Producer notify: Producing an item
[29: 8] ->                                          Producer notify: Produced item number 568
[29: 8] ->                                          Producer notify: Waiting to access production bay
[29: 8] ->                                          Producer notify: Entering production bay
[29: 8] ->                                          Producer notify: Producing an item
[29: 8] ->                                          Producer notify: Produced item number 14
[29: 8] ->                                          Producer notify: Waiting to access production bay
[29: 8] ->                                          Producer notify: Entering production bay
[29: 8] ->                                          Producer notify: Producing an item
[29: 9] ->                                          Producer notify: Produced item number 122
[29: 9] ->                                          Producer notify: Waiting to access production bay
[29: 9] ->                                          Producer notify: Entering production bay
[29: 9] ->                                          Producer notify: Producing an item
[29: 9] ->                                          Producer notify: Produced item number 1
[29: 9] ->                                          Producer notify: Waiting to access production bay
[29: 9] ->                                          Producer notify: Entering production bay
[29: 9] ->                                          Producer notify: Producing an item
[29: 9] ->                                          Producer notify: Produced item number 508
[29: 9] ->                                          Producer notify: Waiting to access production bay
[29: 9] ->                                          Producer notify: Entering production bay
[29: 9] ->                                          Producer notify: Producing an item
[29: 9] ->                                          Producer notify: Produced item number 706
[29: 9] ->                                          Producer notify: Waiting to access production bay
[29: 9] -> Consumer notify: Entering production bay
[29: 9] -> Consumer notify: Consuming item
[29: 9] -> Consumer notify: Consumed item number 706
[29: 9] -> Consumer notify: Waiting for new items
[29: 9] -> Consumer notify: Entering production bay
[29: 9] -> Consumer notify: Consuming item
^CConsumer won the lock 96 times
Producer won the lock 98 times
[29: 9] -> Consumer notify: Consumed item number 706
[29: 9] ->                                          Producer notify: Entering production bay
[29: 9] ->                                          Producer notify: Producing an item
alumno@ParalelismoMV:~/Compartida/P2-ConcurrenciaBasica$
```

Figura 12: Ejecución del codigo3 con condición de carrera

entonces puede darse el caso de que apenas adquieran el control, o en casos extremos, de que nunca lo hagan. En este último escenario puede darse un problema de inanición, es decir que sus característica le impidan, si no hay una coordinación o control externo, adquirir recursos, en este caso de computación, y aparentar estar 'muertos'.

Tarea 11 – Productores y Consumidores, problema de la inanición

Observa y ejecuta el código "codigo4-ProdConsInanition". Este código también es similar al anterior, pero representa un escenario en el que hay 2 consumidores que buscan entrar en un 'restaurante' y consumir un producto. Sin embargo uno es un **rápido glotón**, y siempre intenta entrar, incluso nada más salir. Mientras el otro es un **lento educado** y además de esperar al salir, si ve a alguien esperando, prefiere volver en otro momento.

- Analiza el código y observa el resultado de su ejecución, tienes un ejemplo de la salida en la Figure 13.

Inanición (codigo4-ProdConsInanition.py)

```
37 def consumerFast():
38     count = 0
39     while True:
40         fast_notify("Waiting to enter diner")
41         item_ready.acquire()
42         fast_notify("Entering diner and eating")
43         time.sleep(EAT_TIME)
44         count += 1
45         fast_notify("Exiting diner, I have eaten {0} times".format(count))
46         item_ready.release()
47         fast_notify("Going to queue to eat again")
48
49 def consumerSlow():
50     count = 0
51     while True:
52         slow_notify("Waiting to enter diner")
53         success = item_ready.acquire(timeout=0.1) # Operación no-bloqueante al usar un timeout
54         # Se guarda el resultado en success, si se adquirió el lock, sera True, sino será False
55         if not success:
56             slow_notify("Someone is inside, going to make time")
57             time.sleep(1)
58             continue
59         slow_notify("Entering diner and eating something")
60         time.sleep(EAT_TIME)
61         count += 1
62         slow_notify("Exiting diner, I have eaten {0} times".format(count))
63         slow_notify("Going for a brief walk")
64         item_ready.release()
65         time.sleep(1)
```

```
[31:17] -> Launching threads
[31:17] -> Fast consumer: Waiting to enter diner
[31:17] -> Fast consumer: Entering diner and eating
[31:17] ->                                                    Slow consumer: Waiting to enter diner
[31:17] -> Fast consumer: Exiting diner, I have eaten 1 times
[31:17] -> Fast consumer: Going to queue to eat again
[31:17] -> Fast consumer: Waiting to enter diner
[31:17] -> Fast consumer: Entering diner and eating
[31:17] ->                                                    Slow consumer: Someone is inside, going to make time
[31:17] -> Fast consumer: Exiting diner, I have eaten 2 times
[31:17] -> Fast consumer: Going to queue to eat again
[31:17] -> Fast consumer: Waiting to enter diner
[31:17] -> Fast consumer: Entering diner and eating
[31:17] -> Fast consumer: Exiting diner, I have eaten 3 times
[31:17] -> Fast consumer: Going to queue to eat again
[31:17] -> Fast consumer: Waiting to enter diner
[31:17] -> Fast consumer: Entering diner and eating
[31:17] -> Fast consumer: Exiting diner, I have eaten 4 times
[31:17] -> Fast consumer: Going to queue to eat again
[31:17] -> Fast consumer: Waiting to enter diner
[31:17] -> Fast consumer: Entering diner and eating
[31:17] -> Fast consumer: Exiting diner, I have eaten 5 times
[31:17] -> Fast consumer: Going to queue to eat again
[31:17] -> Fast consumer: Waiting to enter diner
[31:17] -> Fast consumer: Entering diner and eating
[31:17] -> Fast consumer: Exiting diner, I have eaten 6 times
[31:17] -> Fast consumer: Going to queue to eat again
[31:17] -> Fast consumer: Waiting to enter diner
[31:17] -> Fast consumer: Entering diner and eating
[31:17] -> Fast consumer: Exiting diner, I have eaten 7 times
[31:17] -> Fast consumer: Going to queue to eat again
[31:17] -> Fast consumer: Waiting to enter diner
[31:17] -> Fast consumer: Entering diner and eating
[31:17] -> Fast consumer: Exiting diner, I have eaten 8 times
[31:17] -> Fast consumer: Going to queue to eat again
```

Figura 13: Ejecución del codigo4 con inanición de uno de los hilos.

Timeouts: Muchas operaciones que son 'bloqueantes', como la adquisición de un lock, pueden ser configuradas con un *timeout*, un tiempo máximo de espera después del cual se continuará con la ejecución. Para distinguir si la operación fue exitosa, es habitual usar el valor retornado o capturar una excepción. Puedes ver esto en la línea 53 del código3.

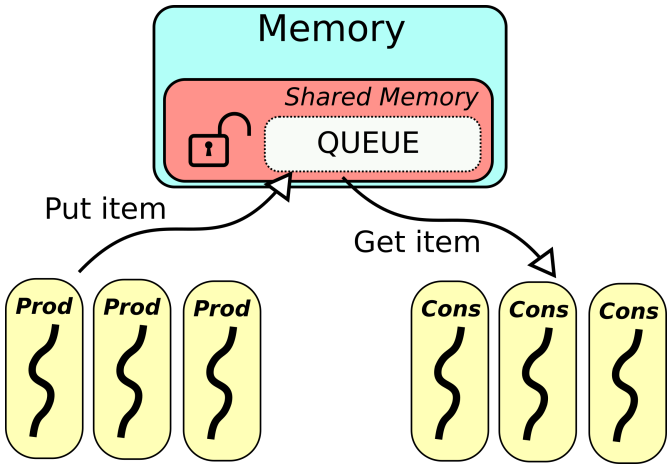

Figura 14: Representación gráfica de hilos compartiendo datos mediante una cola

4. Múltiples productores y consumidores

Si generalizamos los códigos anteriores podemos explorar el problema de productores y consumidores con un número mayor de ambos, algo que se puede ver en el *'codigo5-MultProdCons.py'*. No obstante, en este escenario no vamos a usar variables para coordinar los hilos ya que al tener más de 1 hilo de cada tipo, vamos también a tener un número mayor de 1 de 'productos' a consumir, que a su vez podrán ser consumidos por más de 1 potencial consumidor. De seguir la lógica de los programas anteriores, tendríamos que definir una estructura de datos donde deberíamos guardar los productos producidos, pero también sus locks asociados, y a su vez deberíamos programar la lógica necesaria.

En lugar de esto vamos a usar una estructura de datos que ya es *'thread-safe'*, es decir, cuya coherencia ya está gestionada por la librería. La estructura que vamos a usar es una *cola* (Documentación para Python). Esta estructura tiene un uso similar a una lista de elementos, pudiendo añadir o quitar elementos de ella bajo necesidad, pero su implementación nos asegura que dichas operaciones serán llevadas a cabo de forma segura, incluso cuando varios hilos intenten añadir o eliminar elementos de forma 'simultánea'.

Información: Definimos algo como *'thread-safe'* cuando puede ser usado por varios hilos, sin necesidad de que el programador o el usuario implemente ninguna lógica de control. Por ejemplo, la variable que vimos en el Escenario 3 de la primera sección (Diferencia entre Hilos y Procesos) es 'thread-safe'. De igual forma, la implementación que hemos visto en los códigos anteriores también es thread-safe, aunque en algunos casos presentara problemas por otros motivos ya descritos. También se pueden calificar estructuras como listas o diccionarios como 'thread-safe'.

4.1. Programación de los códigos

En este punto es interesante explicar 3 'estilos' o tipos de segmentos de código que serán repetidos en las siguientes prácticas, y que serían:

- **Lógica:**
  - Código que implementa la lógica del problema a resolver con el programa.
  - Este código cambiará según el problema estudiado en la práctica.
  - En este caso → función *decrease_counter*, clases *ProducerThread* y *ConsumerThread*

Código de lógica, hilo Consumer (codigo5-MultProdCons.py)

```
class ConsumerThread(Thread):
    def run(self):
        global queue, items_pending, OUTPUT
        while True:
            sleep_time = random.randint(MIN_SLEEP_CONSUMER, MAX_SLEEP_CONSUMER)
            try:
                num = queue.get(block=QUEUE_BLOCK)
                # Si la cola está vacía, se lanza la excepción "Empty"
                decrease_counter()
                if OUTPUT:
                    print("[-] CONSUMER {0} item number {1:3d}, "
                          "sleeping {2} miliseconds QUEUE : [{3}/{4}] ITEMS: "
                          "[{5} pending]".format(self.native_id, num,
                                                 sleep_time, queue.qsize(),
                                                 QUEUE_SIZE, int(items_pending)))
            except Empty:
                pass
                # La cola estaba vacía, no se hace nada -> pass

            time.sleep(sleep_time / 1000)

            if items_pending <= 0:
                if OUTPUT:
                    print("[*] CONSUMER {0} --> FINISHING".format(self.native_id))
                break
```

Código de lógica, función '*decrease_counter*' (codigo5-MultProdCons.py)

```
def decrease_counter():
    global items_pending
    counter_lock.acquire()
    items_pending -= 1
    counter_lock.release()
```

- **Paralelización:**
  - Código que implementa la parte más técnica de la paralelización del problema.
  - Este tipo de código se repetirá a lo largo de las prácticas ya que las técnicas para paralelizar serán similares o reusadas.
  - En este caso → función *spawn*

Código de paralelización, función '*spawn*' (codigo5-MultProdCons.py)

```
def spawn():
    consumers_list = list()
    for _ in range(0, NUM_CONSUMERS):
        c = ConsumerThread()
        consumers_list.append(c)

    producers_list = list()
    for _ in range(0, NUM_PRODUCERS):
        p = ProducerThread()
        producers_list.append(p)

    for p, c in zip(consumers_list, producers_list):
        p.start()
        c.start()

    start = time.time()
    for p, c in zip(consumers_list, producers_list):
        c.join()
        p.join()
    end = time.time()
    return to_time_miliseconds(start, end)
```

- **Experimentación:**
  - Este código puede variar entre prácticas, pero la idea será la misma siempre, variables globales y/o pasadas como argumento que configurarán el experimento, así como funciones auxiliares que gestionarán tiempos o guardarán y etiquetarán los resultados obtenidos.
  - En este caso → funciones *save_config*, *run_experiment* y el propio *main* del programa.

Código de paralelización, función '*run_experiment*' (codigo5-MultProdCons.py)

```
def run_experiment():
    global i, measurements, items_pending, queue
    queue = Queue(QUEUE_SIZE)
    items_pending = ITEMS_TO_PROCESS
    measurements[i] = save_config()
    measure = spawn()
    measurements[i]["time"] = measure
    print("Took {0} miliseconds to produce and consume {1} items".format(measure, int(ITEMS_TO_PROCESS)))
    i += 1
```

Atención: Recuerda el papel crítico de las variables globales a la hora de configurar los experimentos. En este caso, por ejemplo las variables como:

- **NUM_CONSUMERS** y **NUM_PRODUCERS** determinan el *grado de concurrencia*, ya que serán las que dirán cuantos hilos se lanzarán de cada tipo.
- **ITEMS_TO_PROCESS** determinan la *duración del experimento*, a más 'items' a procesar, en general más tardará la ejecución de un experimento, aunque el tiempo final dependerá del resto de variables.
- **QUEUE_SIZE** determina en general el *tamaño de los recursos compartidos* y usados para la coordinación y paso de datos, en este caso de la cola. En general, a menor tamaño, más contención por esos recursos y así, más tiempo de espera y bloqueo. Si el tamaño es muy grande por un lado se podrá tener un mayor gasto de memoria, y por otro a partir de un punto, no se notarán mejoras en el rendimiento. Una cola de tamaño 0 es 'infinita' (hasta que se acabe la memoria libre).
- **MIN_SLEEP_PRODUCER** o **MAX_SLEEP_PRODUCER** determinan el tiempo que permanecerán dormidos procesos o hilos. Estos tiempos podrán ser en milisegundos o en segundos, pero de ser muy cortos pueden tratarse como 'no dormir' o si por error los configuramos como muy largos, podrían ser de minutos (demasiado).

4.2. Experimentación

Tarea 12 – Múltiples Productores y Consumidores, número de productores/consumidores

Empezando con el análisis del impacto de 1 tipo de variable, en este caso de **NUM_CONSUMERS** y **NUM_PRODUCERS**. Ejecuta el **"EXP 1"**, que analiza el efecto del ratio entre el número de productores y de consumidores. Observa que para este experimento se han fijado una serie de variables con unos valores concretos.

- ITEMS_TO_PROCESS = 1e2
- QUEUE_SIZE = 10
- MIN_SLEEP_CONSUMER, MAX_SLEEP_CONSUMER = 250, 500
- MIN_SLEEP_PRODUCER, MAX_SLEEP_PRODUCER = 250, 500

Y se ha iterado con otros

- NUM_CONSUMERS, NUM_PRODUCERS = [(5, 10), (10, 5), (10, 10), (100, 100)]

Ejecuta el experimento y observa la salida:

- **¿Afecta el ratio de productores a consumidores? ¿Por qué?**
- **¿Cómo evolucionan los tiempos cuando se aumentan de forma equilibrada el número de productores y consumidores?**

Tarea 13 – Múltiples Productores y Consumidores, tiempos mínimos y máximos

Ejecuta ahora el segundo experimento (**"EXP 2"**), que estudia el efecto de los tiempos mínimos y máximos para los productores y consumidores. En este caso las variables fijadas y las que analizaremos serán otras.

- **¿Qué efecto tienen las esperas mínimas y máximas de los consumidores y los productores?**

Tarea 14 – Múltiples Productores y Consumidores, tamaño de la cola

Ejecuta ahora el segundo experimento (**"EXP 3"**), que estudia el papel que tiene la cola.

- **¿Tiene efecto el tamaño de la cola?**

Tarea 15 – Múltiples Productores y Consumidores, tamaño de la carga

Finalmente, ejecuta ahora el (**"EXP 4"**), que analiza como varían los tiempos a medida que aumenta el número de ítems a procesar.

- **¿Cómo aumentan dichos tiempos a medida que el número de items se incrementa?**

```
## EXP 1 --> Efecto del ratio de productores y consumidores ##
Took 8157 miliseconds to produce and consume 100 items
Took 8190 miliseconds to produce and consume 100 items
Took 8533 miliseconds to produce and consume 100 items
Took 4433 miliseconds to produce and consume 100 items
Took 791 miliseconds to produce and consume 100 items

                        0     1     2     3    4
-----------------------------------------------------
ITEMS_TO_PROCESS      100   100   100   100  100
-----------------------------------------------------
QUEUE_SIZE             10    10    10    10   10
-----------------------------------------------------
MIN_SLEEP_CONSUMER    250   250   250   250  250
MAX_SLEEP_CONSUMER    500   500   500   500  500
NUM_CONSUMERS           5     5    10    10  100
-----------------------------------------------------
MIN_SLEEP_PRODUCER    250   250   250   250  250
MAX_SLEEP_PRODUCER    500   500   500   500  500
NUM_PRODUCERS           5    10     5    10  100
-----------------------------------------------------
time                 8157  8190  8533  4433  791
######
```

(a) EXP 1

```
## EXP 2 --> Efecto de las esperas mínimas y máximas ##
Took 1978 miliseconds to produce and consume 200 items
Took 3621 miliseconds to produce and consume 200 items
Took 3673 miliseconds to produce and consume 200 items
Took 3779 miliseconds to produce and consume 200 items

                        0     1     2     3
-----------------------------------------------
ITEMS_TO_PROCESS      200   200   200   200
-----------------------------------------------
QUEUE_SIZE             10    10    10    10
-----------------------------------------------
MIN_SLEEP_CONSUMER    250   250   500   500
MAX_SLEEP_CONSUMER    500   500  1000  1000
NUM_CONSUMERS          50    50    50    50
-----------------------------------------------
MIN_SLEEP_PRODUCER    250   500   250   500
MAX_SLEEP_PRODUCER    500  1000   500  1000
NUM_PRODUCERS          50    50    50    50
-----------------------------------------------
time                 1978  3621  3673  3779
######
```

(b) EXP 2

```
## EXP 3 --> Efecto del tamaño de las colas ##
Took 16365 miliseconds to produce and consume 1000 items
Took 11651 miliseconds to produce and consume 1000 items
Took 8767 miliseconds to produce and consume 1000 items
Took 8088 miliseconds to produce and consume 1000 items

                        0      1     2     3
-----------------------------------------------
ITEMS_TO_PROCESS     1000   1000  1000  1000
-----------------------------------------------
QUEUE_SIZE              1      2    10     0
-----------------------------------------------
MIN_SLEEP_CONSUMER    250    250   250   250
MAX_SLEEP_CONSUMER    500    500   500   500
NUM_CONSUMERS          50     50    50    50
-----------------------------------------------
MIN_SLEEP_PRODUCER    250    250   250   250
MAX_SLEEP_PRODUCER    500    500   500   500
NUM_PRODUCERS          50     50    50    50
-----------------------------------------------
time                16365  11651  8767  8088
######
```

(c) EXP 3

```
## EXP 4 --> Efecto del número de items a procesar ##
Took 1205 miliseconds to produce and consume 100 items
Took 1980 miliseconds to produce and consume 200 items
Took 3499 miliseconds to produce and consume 400 items

                        0     1     2
-----------------------------------------
ITEMS_TO_PROCESS      100   200   400
-----------------------------------------
QUEUE_SIZE             10    10    10
-----------------------------------------
MIN_SLEEP_CONSUMER    250   250   250
MAX_SLEEP_CONSUMER    500   500   500
NUM_CONSUMERS          50    50    50
-----------------------------------------
MIN_SLEEP_PRODUCER    250   250   250
MAX_SLEEP_PRODUCER    500   500   500
NUM_PRODUCERS          50    50    50
-----------------------------------------
time                 1205  1980  3499
######
```

(d) EXP 4

Figura 15: Ejecución de los experimentos del codigo5

Copyright & Disclaimer


```
# Copyright © 2025 by Jonatan Enes (jonatan.enes@udc.es)
# Computer Engineering department, Universidade da Coruña, Spain.
#
# This file is part of several courses on parallel processing from Universidade da Coruña,
# and it can only be used and/or modified by the author, the students or any
# explicitly authorized person by the author, inside the context of the subject.
# Redistribution to a third-party without previous authorization is strictly forbidden.
# No commercial use is allowed.
#
# This file has only academic purposes and should not be used for any real-world
# scenario, the author holds no accountability for its use or misuse.
```